

Unit 4 Assignment – Secure Online Banking System (GUI Project)

Overview

In today's digital world, interconnected networks play a vital role in how we live, work, and communicate. Organizations such as banks, hospitals, and schools rely on these systems to manage and protect vast amounts of sensitive data.

As digital information continues to grow, **cybersecurity** has become essential—not only for individuals, but also for businesses and governments. Protecting data from unauthorized access, theft, or corruption is a shared responsibility.

In this assignment, you will take on the role of a **software developer and cybersecurity specialist** to design and build a **Secure Online Banking Application**.

Your Challenge

Design and develop a **Graphical User Interface (GUI) banking system** using **Python (tkinter)** that allows users to perform secure banking operations.

Your application must:

- Use **Object-Oriented Programming (OOP)** principles
- Create and apply **encryption and decryption** to protect sensitive data
- Include **exception handling** for all user inputs
- Provide a clean, intuitive, and interactive user experience

Part I – Graphical User Interface (GUI)

General Requirements

- The entire program must run inside **one main GUI window**
- No interaction should occur in the Python Shell
- The interface must be **user-friendly and well-organized**

1. User Registration System

Create a registration feature that allows users to:

- Enter:
 - Full Name
 - Username

- Password
 - Initial Balance
- Save user(s) data in **encrypted file(s)**, this is your design choice.
- Ensure all stored data is **securely encrypted**

2. User Login System

- Allow users to log in using their credentials
- Retrieve and **decrypt stored data**
- If login is successful, display the **Banking Dashboard**

3. Main Banking Dashboard

After login, display a new interface with:

- Account Number (one per user)
- Current Date
- Options to:
 - Change Password
 - Log Out / Exit
 - Select Account Type:
 - Checking Account
 - Savings Account

4. Checking Account Features

Display the user's full name and allow them to:

- View current balance
- Withdraw money
- Deposit money
- Transfer funds (to savings account)
- Return to main menu
- Exit program

5. Savings Account Features

Display the user's full name and allow them to:

- View:
 - Current balance
 - Total interest earned
- Transfer funds (to checking account)
- Apply **compound interest (calculated per minute)**

- Return to main menu
- Exit program

Part II – Data Classes (OOP Design)

You must design appropriate classes to support your system.

Requirements:

- Apply **Encapsulation**
 - Protect sensitive data using private attributes
- Apply **Inheritance**
 - Example: Base `Account` class → `CheckingAccount` and `SavingsAccount`
- Organize your program logically and efficiently

Part III – Error Handling

Your program must handle all invalid inputs gracefully, including:

- Non-numeric input for transactions
- Negative values
- Insufficient funds
- Incorrect login credentials

Use **try-except blocks** to prevent crashes and guide the user.

Part IV – Cybersecurity (Encryption)

- Encrypt all user data before saving it
- Decrypt data only when needed
- Ensure:
 - Passwords and user information are not readable without decryption
 - File names are not readable without decryption

Part V – Animation Requirement

Enhance your application by including:

- A **custom animated logo** (created using code, NOT GIFs)
- Real-time updates:
 - Savings account balance updates automatically per use as interest accumulates per minute.

Important Notes

- Review the **rubric carefully** before starting
- Use only concepts covered in class unless approved by your teacher
- Any unsupported advanced techniques may **not be graded**

Criteria	Poor (1)	Fair (2)	Good (3)	Great (4)
Requirements: [16 marks]	4+ requirement analysis errors.	2 or 3 requirement analysis errors.	1 requirement analysis error.	All requirements are implemented correctly!
Program Design, Efficiency, and Structure. Organizing your code into modules and classes, OOP concepts. [4 points]	Poor code organization and/or inefficient code.	Fair code organization and/or efficiency.	Good code organization and/or efficiency.	Excellent code organization and efficiency!
User Interface Design [4 points] - Animation - Colour and Layout	Poor	Fair	Good	Excellent
Style Rules [4 points] (#1 - #11)	4+ style rules are broken or need improvement.	2 or 3 style rules are broken or need improvement.	1 style rule broken or needs improvement.	Perfect style! All style rules have been followed.
Overall Mark:				/28